

Enterprise IT Lab

Project Documentation

Ibnousoufyane Ayman

July 2026

github.com/Ayman6594/AD-Project

Skills Demonstrated

Domain	What Was Built
Sysadmin	Active Directory, GPO, DHCP, DNS, OU structure, domain users
Endpoint	Windows 11 domain join, GPO enforcement verified on workstation
Monitoring	Prometheus + Grafana + Node Exporter, live dashboard
SIEM	Graylog + NXLog, centralized Windows event log collection
Security	Brute-force alerting on Event ID 4625, HTTP webhook in real time
VPN	ocserv SSL VPN + PAM-LDAP authentication against Active Directory
IaC	PowerShell automation script, bash install script, GPO software deploy

This document covers sections 1 through 6, documenting every component built, tested, and verified in this lab environment running on VMware Workstation.

Table of Contents

Lab Architecture Overview

1. Active Directory

- Domain Information
- Automation Script
- Organizational Units and Security Groups
- Domain Users Created
- GPOs Deployed

2. Client-01 — Domain Workstation

- Domain Join Steps
- GPO Verification

3. Monitoring — Prometheus + Grafana

- Ubuntu Server Installation
- Installation Script
- Grafana Configuration
- Metrics Collected

4. Graylog — Centralized Log Management

- Architecture
- Docker Compose Deployment
- NXLog Installation
- Automated GPO Deployment
- Results

5. Graylog Alerts — Brute-Force Detection

- Stream Failed-Logons
- Event Definition
- Webhook Notification
- Test Results

6. VPN SSL + AD/LDAP Integration

- ocserv Installation
- Certificate Generation
- Configuration
- LDAP Integration
- Connection Test

What's Next

Conclusion

Lab Architecture Overview

A fully functional enterprise IT environment running on VMware Workstation. Four virtual machines communicate over a Host-Only internal network with internet access via NAT. The lab replicates real-world corporate infrastructure as found in companies across Morocco and internationally.

Virtual Machines and Network

VM	OS	IP Address	RAM	Role
DC01	Windows Server 2022	192.168.1.10	4 GB	Domain Controller, DNS, DHCP
Client-01	Windows 11	192.168.1.x (DHCP)	4 GB	Domain workstation
Monitor-01	Ubuntu 24.04 LTS	192.168.1.101	4 GB	Prometheus, Grafana, Graylog, VPN

Services and Access URLs

Service	URL / Address	Credentials
Grafana	http://192.168.1.101:3000	admin / User@123456
Prometheus	http://192.168.1.101:9090	No auth
Node Exporter	http://192.168.1.101:9100/metrics	No auth
Graylog	http://192.168.1.101:9000	admin / admin
ocserv VPN	192.168.1.101:443 (TCP/UDP)	AD credentials (PAM-LDAP)
DC01 RDP	192.168.1.10	IB\Administrateur
Domain Login	IB\username	User@123456

1. Active Directory

Windows Server 2022 was installed and promoted as the Domain Controller for the IB.local forest. The entire configuration was automated using a single PowerShell script — from OU structure to GPO deployment — demonstrating infrastructure-as-code in a Windows environment.

Domain Information

- : Domain name: IB.local
- : NetBIOS name: IB
- : Forest and domain functional level: Windows Server 2016
- : Domain Controller: DC01 at 192.168.1.10
- : DNS: integrated with AD, pointing to 127.0.0.1

Automation Script

A PowerShell script (AD-Setup.ps1) was written to deploy the full AD configuration automatically. It runs in under 5 minutes and creates the complete environment from scratch.

```
Set-ExecutionPolicy Unrestricted -Force  
.\AD-Setup.ps1
```

What the script creates:

- : 6 Organizational Units: IB-Company, IT, Finance, Management, Users, Computers, Service-Accounts
- : 4 Security Groups: GRP-IT, GRP-Finance, GRP-Management, GRP-AllUsers
- : 5 Domain Users assigned to their respective OUs and groups
- : DHCP Server installed, authorized, and scope configured
- : 6 GPOs created and linked to their target OUs

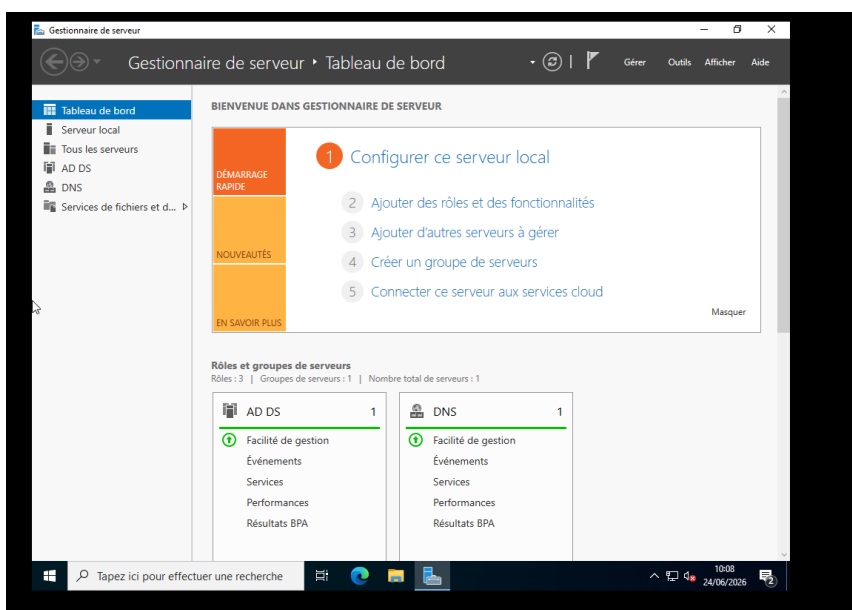


Figure 1 : Server Manager showing AD DS and DNS services active and green

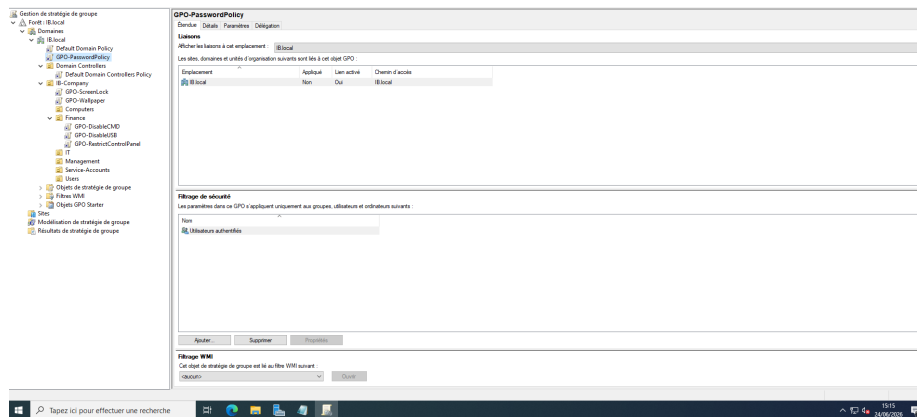


Figure 2 : PowerShell script running — OUs, Groups and Users created automatically

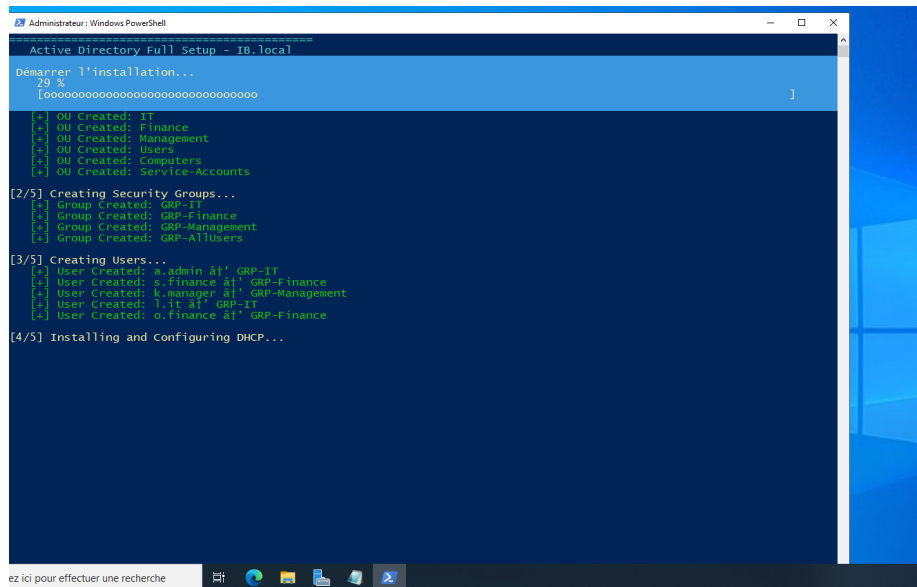


Figure 3 : Group Policy Management Console showing all 6 GPOs linked to their OUs

Users Created

Username	Full Name	OU	Group	Role
a.admin	Ayman Admin	IT	GRP-IT	IT Administrator
l.it	Leila IT	IT	GRP-IT	IT Support
s.finance	Sara Finance	Finance	GRP-Finance	Financial Analyst
o.finance	Omar Finance	Finance	GRP-Finance	Accountant
k.manager	Karim Manager	Management	GRP-Management	Department Manager

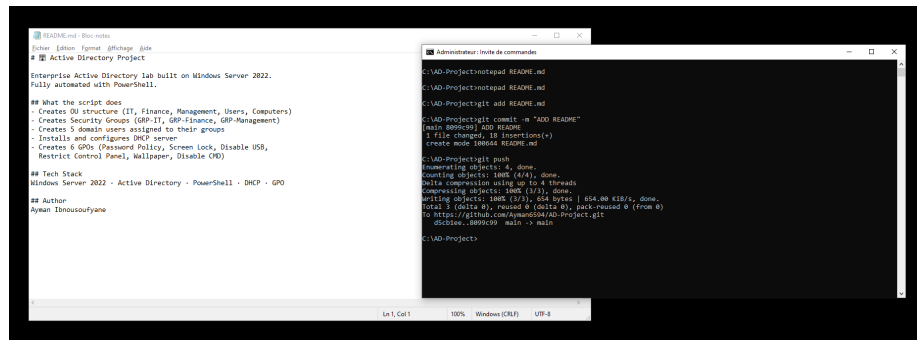


Figure 4 : Full OU and GPO tree structure in IB.local domain

GPOs Deployed

GPO Name	Target	Effect
GPO-PasswordPolicy	IB.local	Strong password + 90-day expiry
GPO-ScreenLock	IB-Company	Screen locks after 10 minutes of inactivity
GPO-Wallpaper	IB-Company	Company wallpaper pushed to all machines
GPO-DisableUSB	Finance OU	USB storage devices blocked
GPO-RestrictControlPanel	Finance OU	Control Panel inaccessible
GPO-DisableCMD	Finance OU	CMD and PowerShell blocked

2. Client-01 — Domain Workstation

A Windows 11 virtual machine was joined to the IB.local domain. Domain users can log in from the workstation and all GPOs apply automatically at login — exactly as in a real corporate environment. The machine was given a Host-Only network adapter so it can reach DC01 at 192.168.1.10.

Domain Join Steps

- : Added Host-Only network adapter in VMware settings
- : Set DNS to 192.168.1.10 (DC01) on the Host-Only adapter
- : System Properties > Change > Domain: IB.local
- : Authenticated with IB\Administrateur — restart
- : Logged in with IB\a.admin and IB\s.finance — confirmed working

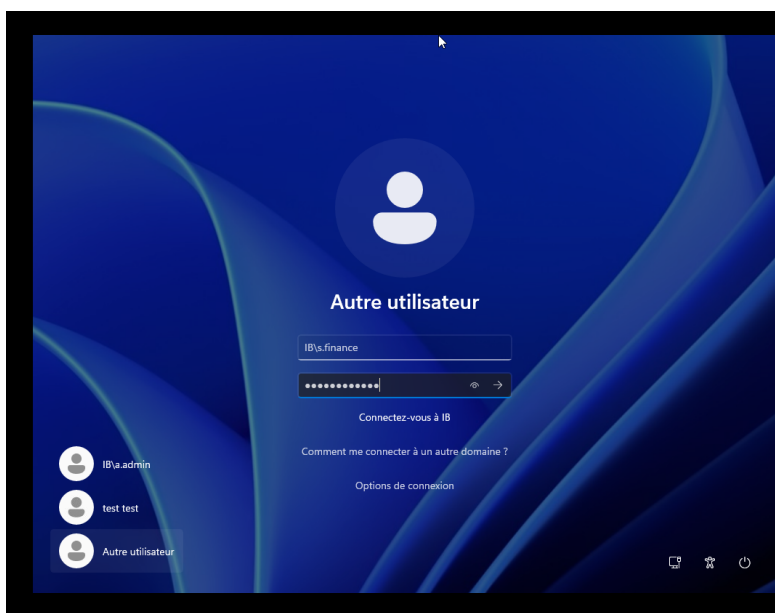


Figure 5 : Windows 11 login screen — IB domain users visible (IB\a.admin and IB\s.finance)

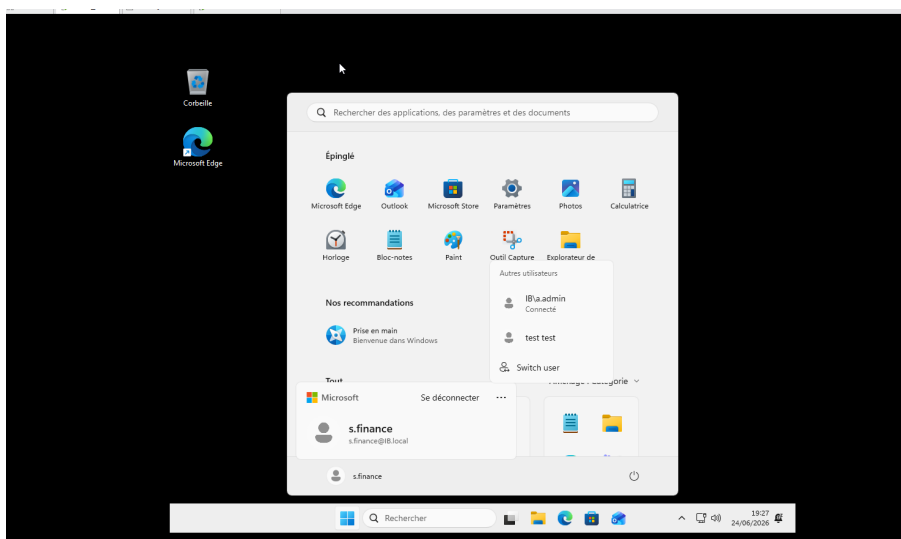


Figure 6 : Start menu logged in as s.finance@IB.local — domain account confirmed

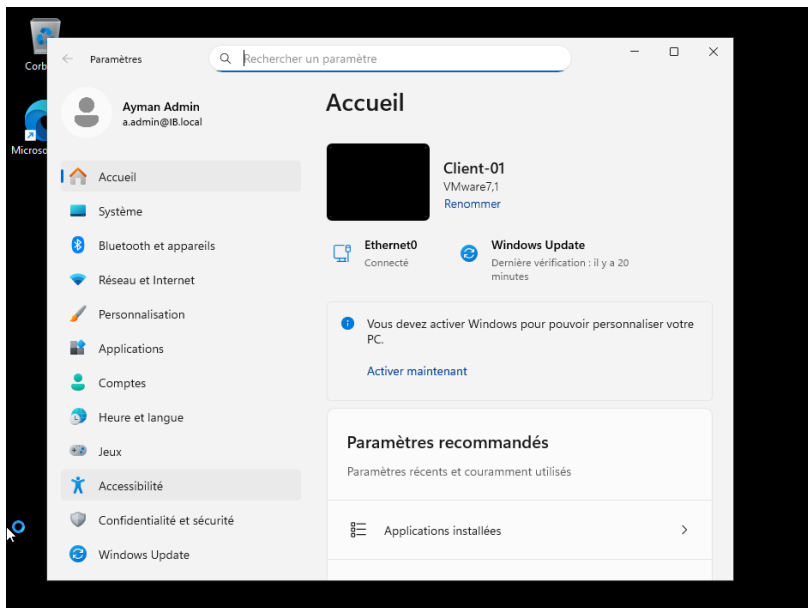


Figure 7 : System Settings showing a.admin@IB.local on Client-01 (machine: Client-01)

GPO Verification

After logging in as s.finance (Finance group), all Finance-targeted GPOs were tested and confirmed working on Client-01.

- : CMD opened: immediately shows "disabled by your administrator"
- : Control Panel: "Operation cancelled due to restrictions on this computer"
- : USB storage device: blocked by the system

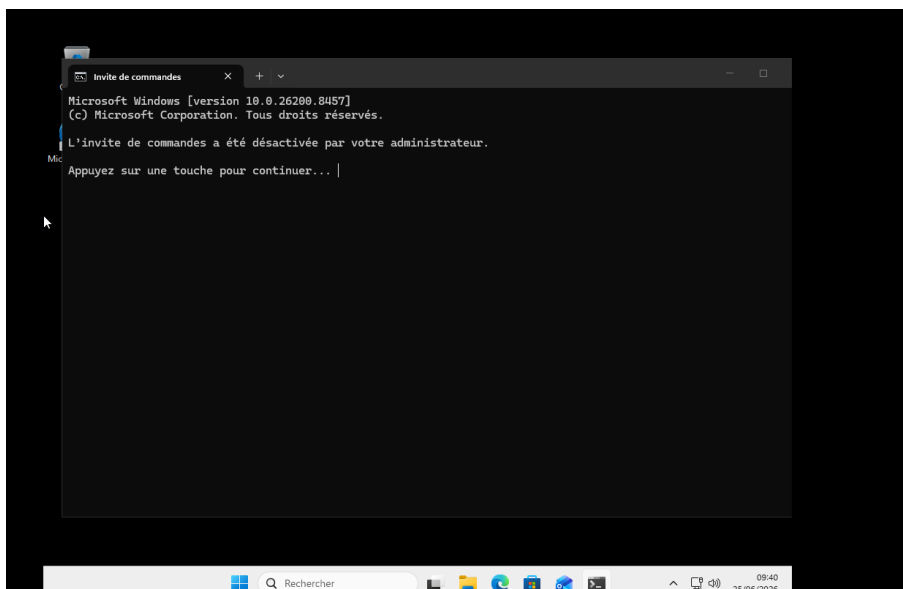


Figure 8 : CMD blocked — 'The command prompt has been disabled by your administrator'

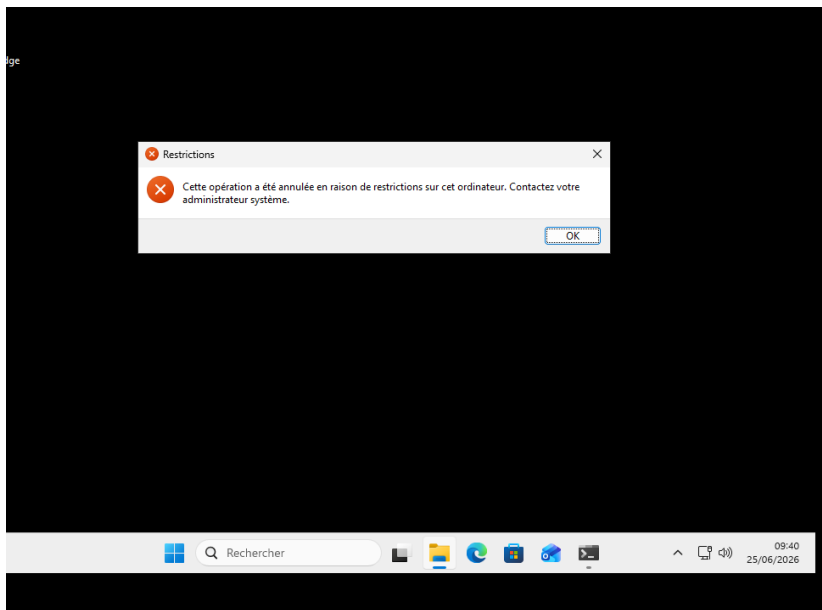


Figure 9 : Control Panel blocked — 'Operation cancelled due to restrictions'

3. Monitoring — Prometheus + Grafana

An Ubuntu 24.04 LTS server (Monitor-01) was deployed as a dedicated monitoring node. Using a single bash script, the full monitoring stack was installed and configured: Prometheus collects metrics every 15 seconds, Node Exporter exposes Linux system stats, and Grafana displays everything in a live dashboard accessible from any machine in the lab.

Ubuntu Server Installation

- : Ubuntu 24.04 LTS ISO downloaded from ubuntu.com/server
- : New VM created in VMware: 4GB RAM, 60GB disk, NAT + Host-Only adapters
- : Static IP set via netplan: 192.168.1.101 on ens34 (Host-Only)
- : Ping to DC01 (192.168.1.10) confirmed — network working

Netplan Configuration

```
sudo nano /etc/netplan/00-installer-config.yaml
sudo chmod 600 /etc/netplan/00-installer-config.yaml
sudo netplan apply
ping 192.168.1.10
```

Installation Script

A bash script (install-monitoring.sh) installs and configures everything automatically. It creates system users, downloads binaries, writes systemd service files, and starts all services.

```
chmod +x ~/install-monitoring.sh
sudo ~/install-monitoring.sh
```

- : Prometheus installed at /usr/local/bin/prometheus — port 9090
- : Node Exporter installed at /usr/local/bin/node_exporter — port 9100
- : Grafana installed via official apt repository — port 3000
- : Windows Exporter installed on DC01 via PowerShell — port 9182

```
monitor-01@monitor-01:~$ ./install-monitoring.sh
libfwupd2 libgusb2
Veuillez utiliser « sudo apt autoremove » pour les supprimer.
Les NOUVEAUX paquets suivants seront installés :
  wazuh-agent
0 mis à jour, 1 nouvellement installés, 0 à enlever et 1 non mis à jour.
Il est nécessaire de prendre 13,2 Mo dans les archives.
Après cette opération, 48,9 Mo d'espace disque supplémentaires seront utilisés.
Réception de :1 https://packages.wazuh.com/4.4/apt/stable/main amd64 wazuh-agent amd64 4.14.5-1 [13,2 MB]
13,2 Mo réceptionnés en 7s (1 969 ko/s)
Préconfiguration des paquets...
Sélection du paquet wazuh-agent précédemment désélectionné.
(Lecture de la base de données... 104100 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../wazuh-agent_4.14.5-1_amd64.deb ...
Dépaquetage de wazuh-agent (4.14.5-1) ...
Paramétrage de wazuh-agent (4.14.5-1) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
Created symlink /etc/systemd/system/multi-user.target.wants/wazuh-agent.service → /usr/lib/systemd/system/wazuh-agent.service.
[+] Wazuh Agent installed

=====
INSTALLATION COMPLETE!
=====

Services running:
Prometheus : http://192.168.1.101:9090
Node Export : http://192.168.1.101:9100
Grafana : http://192.168.1.101:3000
Wazuh Agent : connected to 192.168.1.20

Grafana login:
User : admin
Password: admin

Next: Open Grafana on Client-01 browser!
=====
monitor-01@monitor-01:~$
```

Figure 10 : Bash script output — all components installed and services started

Grafana Configuration

- : Opened at <http://192.168.1.101:3000> from Client-01 browser
- : Default login: admin / admin — changed password on first login
- : Data source added: Prometheus at <http://localhost:9090>
- : Dashboard imported from Grafana.com: Node Exporter Dashboard
- : Instance selected: localhost:9100 — Job: node-exporter

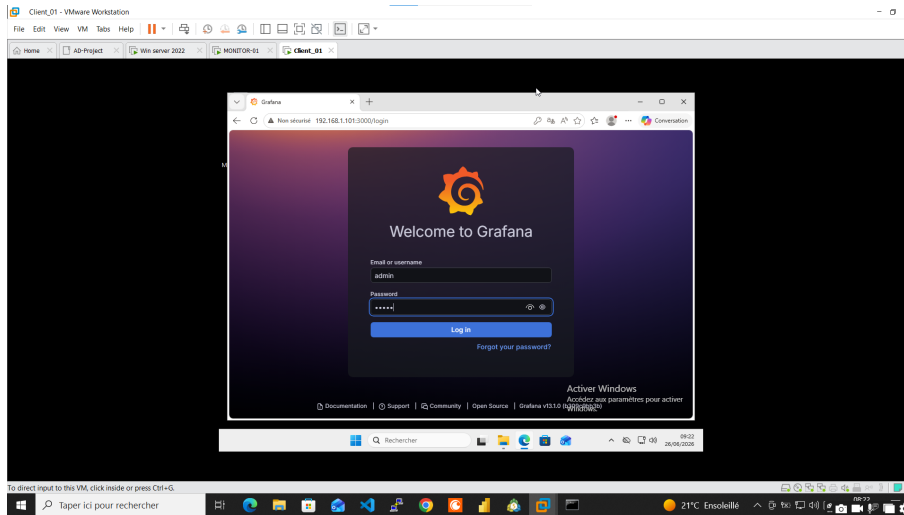


Figure 11 : Grafana login page accessed from Client-01 at <http://192.168.1.101:3000>

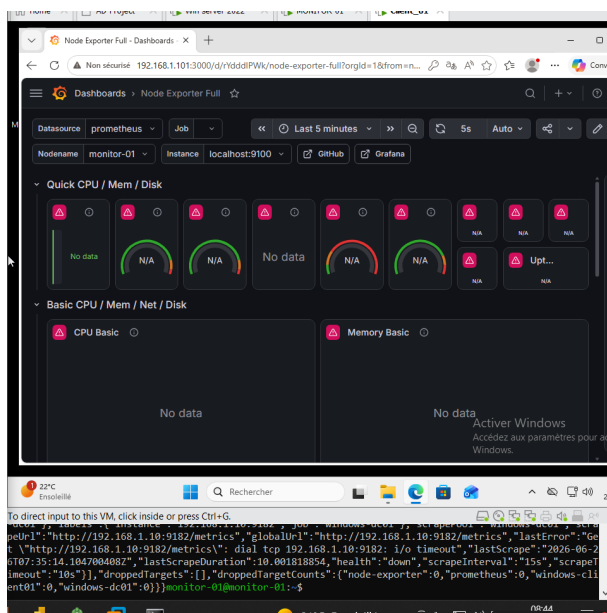


Figure 12 : Dashboard live — CPU 4.9%, RAM 30.9%, Disk 4.9GB used, instance monitor-01

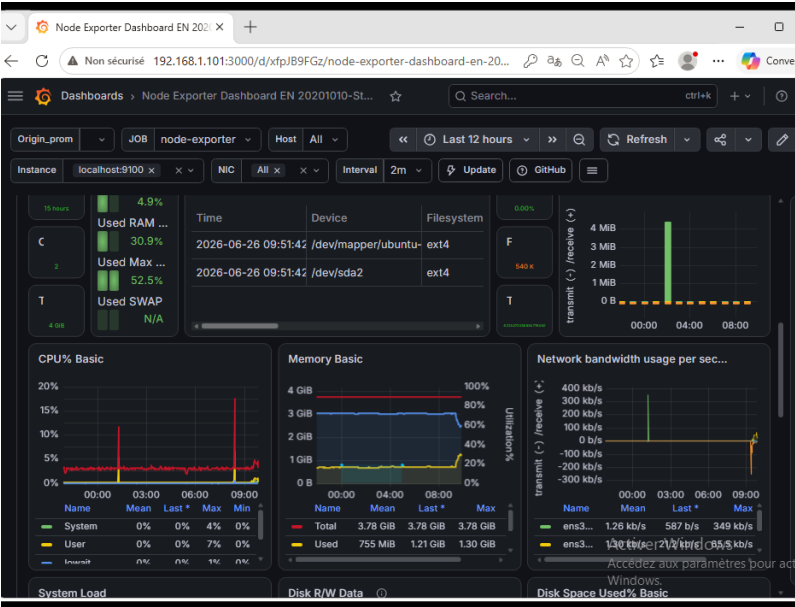


Figure 13 : Full dashboard — CPU Basic, Memory Basic, Network bandwidth, Disk R/W in real-time

Metrics Collected Every 15 Seconds

Metric	Details
CPU usage per core	Idle, user, system, iowait modes
RAM	Total 3.78 GiB — used 1.21 GiB — cached
Disk I/O	/dev/mapper/ubuntuv ext4 and /dev/sda2 read/write
Network bandwidth	ens33 (NAT) and ens34 (Host-Only)
System load average	1m / 5m / 15m
Filesystem usage	Percentage per mount point

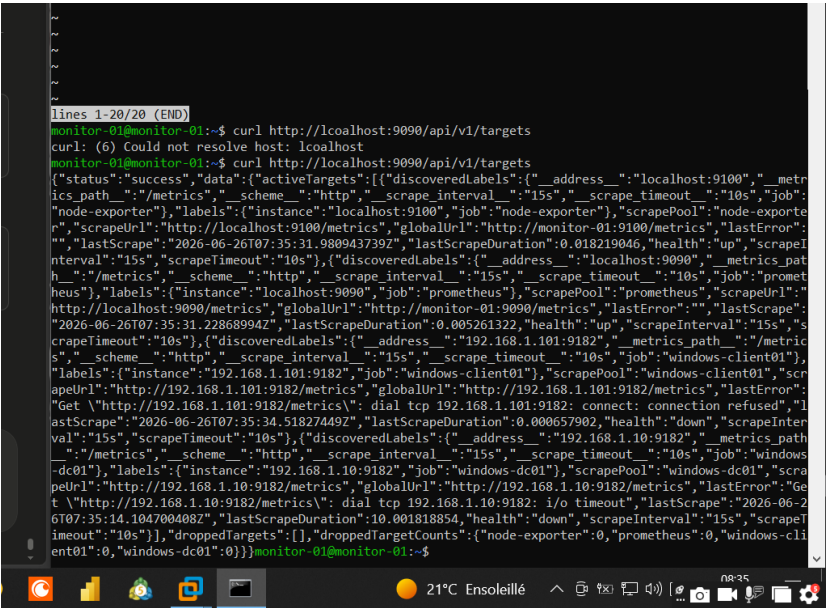


Figure 14 : Prometheus targets — node-exporter UP, Windows Exporter configured

4. Graylog — Centralized Log Management

After encountering severe compatibility issues installing Wazuh SIEM on Ubuntu 24.04 (both the native package and the Docker deployment crashed repeatedly due to API and SSL certificate bugs), the decision was made to switch to Graylog — a simpler and equally professional centralized log management platform widely used in real companies. Graylog collects, indexes, and makes searchable all Windows Event Logs from DC01 and Client-01.

Why Graylog Over Wazuh

- : Wazuh Manager API repeatedly crashed on Ubuntu 24.04 (process exits right after 'Listening')
- : Wazuh Docker deployment failed on certificate mounting (EISDIR errors)
- : Graylog needs only 3 containers (MongoDB, OpenSearch, Graylog) vs 3+ for Wazuh with more SSL complexity
- : Graylog deployed cleanly on the first attempt with Docker Compose

Architecture

Component	Role	Port
MongoDB	Stores Graylog configuration and metadata	27017 (internal)
OpenSearch	Indexes and stores all log messages	9200 (internal)
Graylog	Web UI, REST API, log ingestion (GELF)	9000 (web), 12201/udp (GELF)

Docker Compose Deployment

A docker-compose.yml file defines all three services. Login credentials are pre-configured via environment variables.

```
mkdir -p ~/graylog && cd ~/graylog
nano docker-compose.yml
sudo docker-compose up -d
```

Key environment variables:

```
GRAYLOG_ROOT_PASSWORD_SHA2=8c6976e5b541... # sha256 of "admin"
GRAYLOG_HTTP_EXTERNAL_URI=http://192.168.1.101:9000/
GRAYLOG_ELASTICSEARCH_HOSTS=http://opensearch:9200
GRAYLOG_MONGODB_URI=mongodb://mongodb:27017/graylog
```

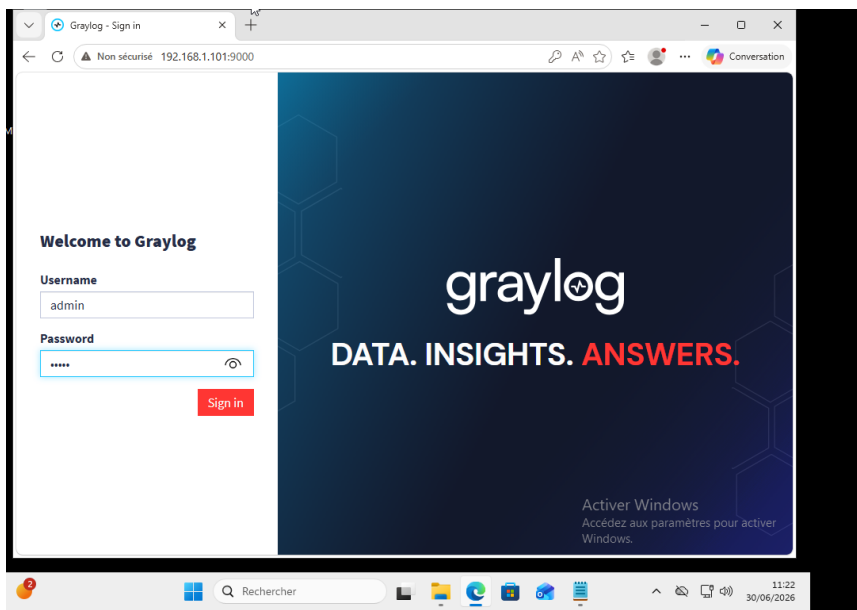


Figure 15 : docker-compose up -d — all 3 containers (mongodb, opensearch, graylog) created successfully

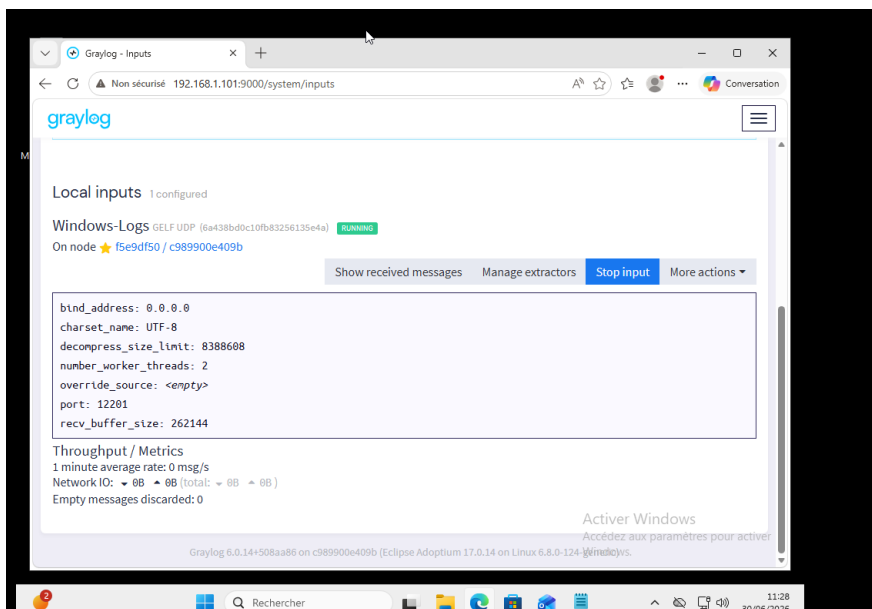


Figure 16 : Graylog Welcome page after first login (admin/admin) from Client-01

GELF Input Configuration

A GELF UDP input was created in Graylog to receive log messages on port 12201. This is the endpoint that NXLog (installed on the Windows machines) forwards Windows Event Logs to.

- : System > Inputs > select "GELF UDP" > Launch new input
- : Title: Windows-Logs | Bind address: 0.0.0.0 | Port: 12201

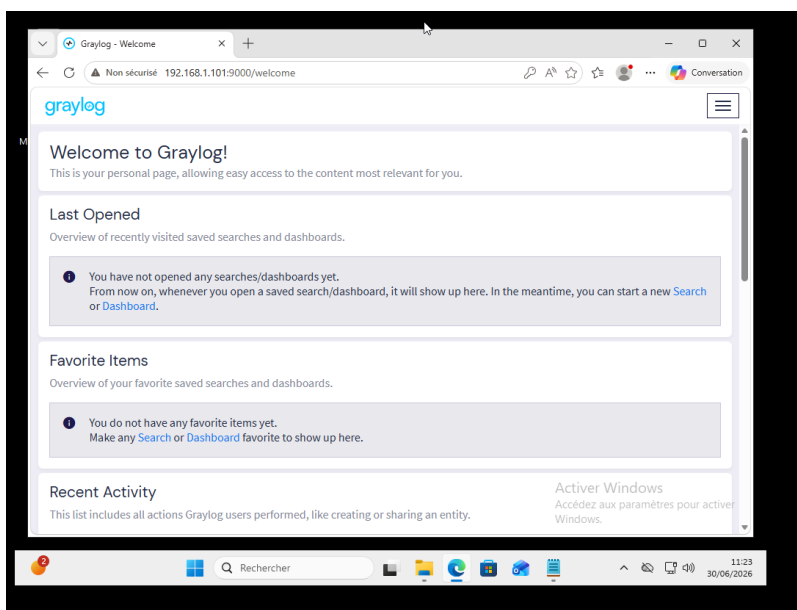


Figure 17 : Windows-Logs GELF UDP input RUNNING on port 12201

Installing NXLog on DC01 and Client-01

NXLog Community Edition was installed on both Windows machines to read the Windows Event Log (Application, System, Security channels) and forward each event to Graylog in GELF format over UDP.

```
notepad "C:\Program Files\nxlog\conf\nxlog.conf"
```

Key configuration block (same on both machines):

```
Module im_msvistalog
Query ...Application/System/Security...

Module om_udp
Host 192.168.1.101
Port 12201
OutputType GELF

Restart-Service nxlog
Get-Service nxlog # Status: Running
```

Automated GPO Deployment (DC01 to Client-01)

To make the deployment repeatable and demonstrate enterprise-style software distribution, a GPO was configured to push the NXLog MSI to all domain computers automatically via Computer Configuration > Software Installation.

```
New-Item -Path "C:\Software" -ItemType Directory -Force
New-SmbShare -Name "Software" -Path "C:\Software" -FullAccess "S-1-1-0"
icacls C:\Software /grant "Authenticated Users:(OI)(CI)F"
icacls C:\Software /grant "Domain Computers:(OI)(CI)F"

New-GPO -Name "GPO-Deploy-NXLog"
New-GPLink -Name "GPO-Deploy-NXLog" -Target "OU=Computers,OU=IB-Company,DC=IB,DC=local"
```

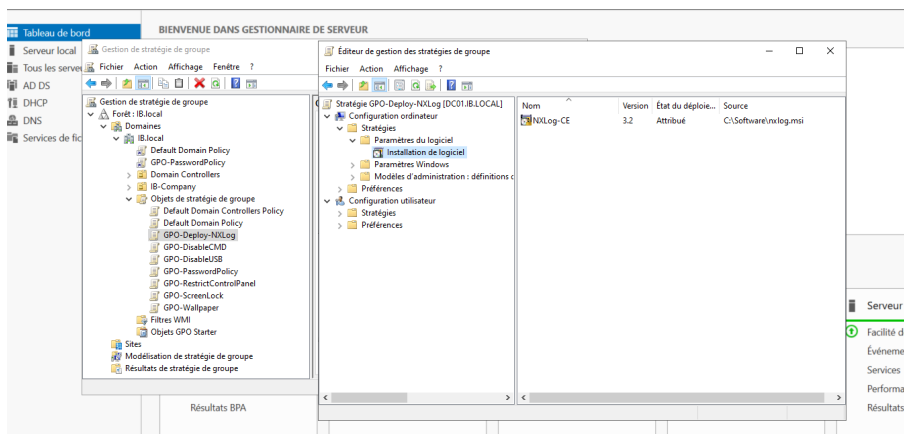


Figure 18 : Software Installation package NXLog-CE assigned via GPO-Deploy-NXLog, source \\DC01\Software\nxlog.msi

Results — Logs Arriving in Real Time

With NXLog running on both DC01 and Client-01, Windows Security and System events (logons, logoffs, special privilege assignments, service state changes) now stream into Graylog continuously and are fully searchable.

- : DC01.IB.local: account logon/logoff, special privileges, service configuration events
- : Client-01.IB.local: system events, network configuration changes
- : Message count graph showing continuous activity, updating live
- : Full-text search across all collected fields

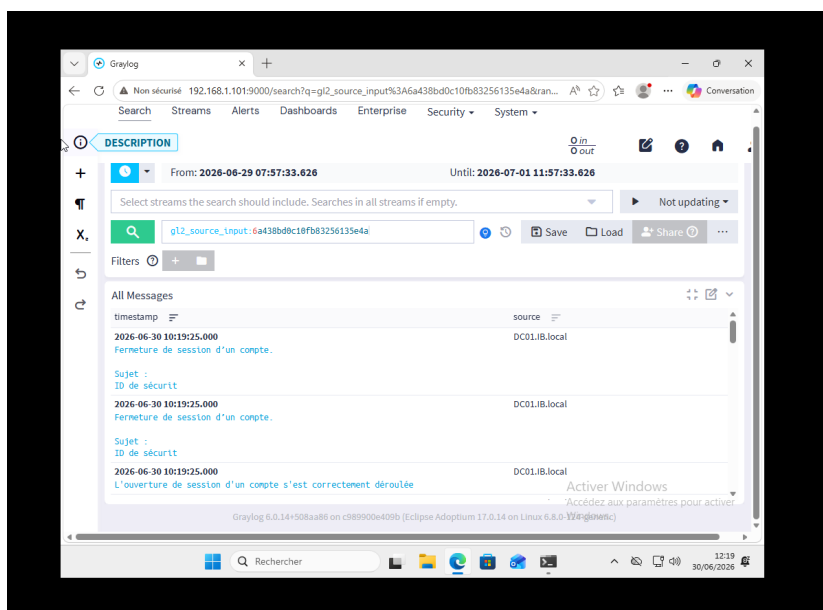


Figure 19 : First logs arriving from DC01.IB.local — account logon/logoff events, security IDs

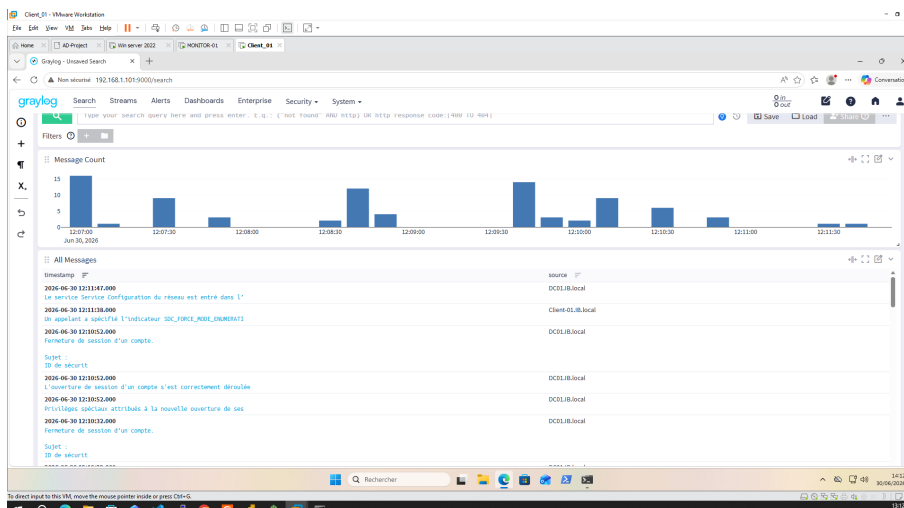


Figure 20 : Logs from BOTH DC01.IB.local and Client-01.IB.local in the same search view, with live message-count graph

5. Graylog Alerts — Brute-Force Detection

An automated alert system was configured to detect brute-force login attempts in real time. When any domain account accumulates more than 5 failed authentication attempts (Windows Event ID 4625) within a 5-minute window, an alert is generated and forwarded to an HTTP webhook receiver running on Monitor-01.

Stream: Failed-Logons

A dedicated Graylog stream was created to filter only Event ID 4625 messages. Since Graylog 6.x stores EventID as an integer, two range rules are used instead of an exact-match rule.

```
Stream name: Failed-Logons
Rule 1: EventID must be greater than 4624
Rule 2: EventID must be smaller than 4626
Mode: Match ALL rules
```

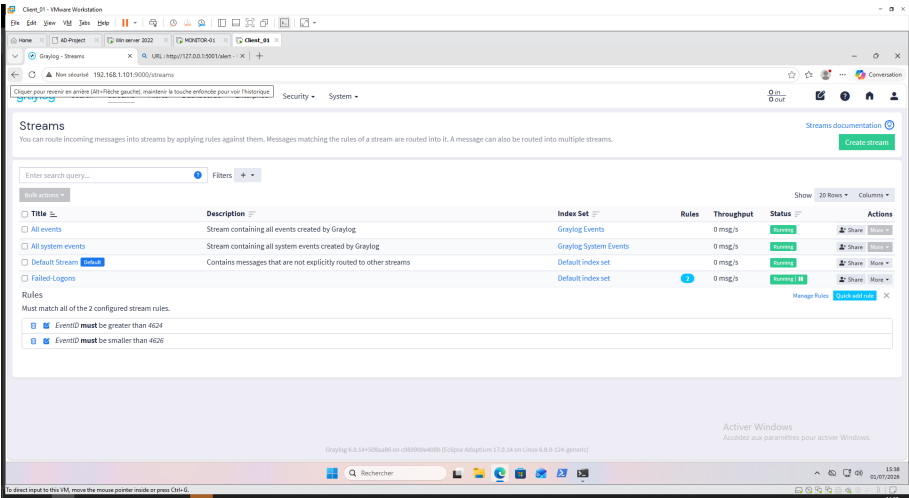


Figure 21 : Stream Failed-Logons Running with 2 rules — EventID range captures exactly 4625

Event Definition

The Event Definition monitors the Failed-Logons stream every minute and triggers when a single account exceeds 5 failed logins within 5 minutes.

Parameter	Value
Title	Brute-Force Detected - 4625
Priority	High
Condition type	Aggregation of results reaches a threshold
Search Query	EventID:4625
Stream	Failed-Logons
Search within last	5 minutes
Execute every	1 minute
Group by field	TargetUserName
Threshold	count() > 4 (triggers when count reaches 5 or more)

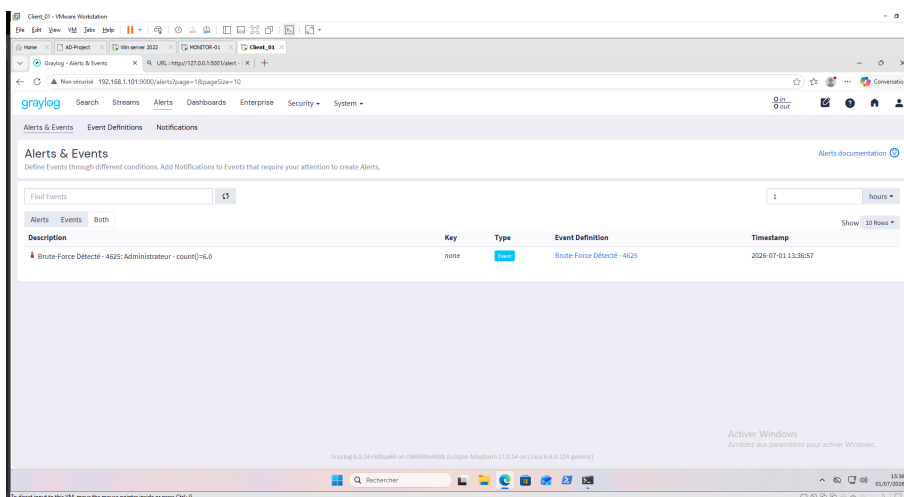


Figure 22 : Alert triggered — Brute-Force Detected - 4625: Administrateur - count()=6.0

Webhook Notification

A Python HTTP server was deployed on Monitor-01 to receive POST notifications from Graylog when an alert fires. The webhook endpoint is registered in Graylog's URL whitelist and linked to the Event Definition.

```
# graylog-webhook.py – runs on Monitor-01 port 5001
from http.server import HTTPServer, BaseHTTPRequestHandler
import json, datetime

class AlertHandler(BaseHTTPRequestHandler):
    def do_POST(self):
        length = int(self.headers['Content-Length'])
        body = json.loads(self.rfile.read(length))
        print(f"[{datetime.datetime.now()}] ALERT RECEIVED")
        print(f" Title : {body.get('event_definition_title', 'N/A')}")
        print(f" Account : {body.get('event', {}).get('message', 'N/A')}")
        self.send_response(200)
        self.end_headers()
    def log_message(self, *args): pass

HTTPServer(('0.0.0.0', 5001), AlertHandler).serve_forever()

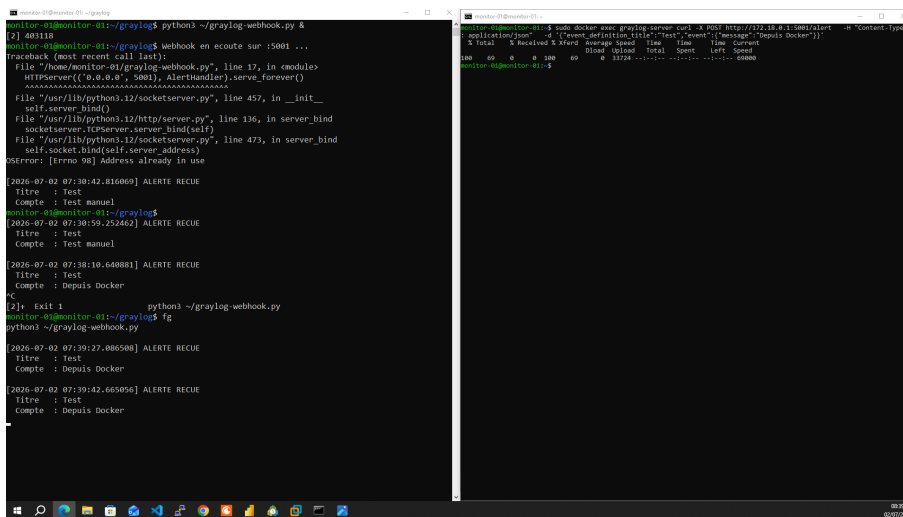
# Start webhook in background
python3 ~/graylog-webhook.py &

# Graylog Notification settings:
# Type: HTTP Notification
# URL: http://172.18.0.1:5001/alert (Docker gateway IP)

# Open port for Docker network:
sudo ufw allow 5001/tcp
sudo ufw allow in on docker0 to any port 5001
```

Test Results

Six failed login attempts were generated on DC01 using incorrect passwords for the Administrateur account. After 1 minute, Graylog's aggregation engine detected count()=6 on TargetUserName=Administrateur and fired the alert to the webhook.



The image shows two terminal windows. The left window is the Monitor-01 terminal, which is running a Python script that listens for HTTP webhooks. It shows several alerts being received, including 'ALERTE RECUE' and 'ALERTE RECUE' with details like 'Titre : Test' and 'Compte : Test manuel'. The right window is the Monitor-01 terminal, which is running a Docker container for Graylog. It shows the Graylog server running and receiving alerts from the Monitor-01 terminal.

Figure 23 : Monitor-01 terminal receiving the Graylog alert via HTTP webhook in real time

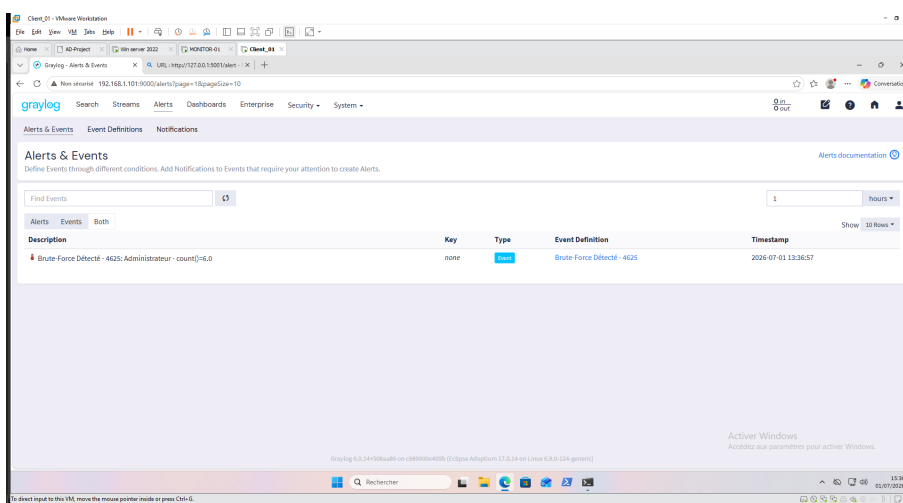


Figure 24 : DC01 generating failed logins (left) — Monitor-01 receiving live alert (right) — count()=6.0

6. VPN SSL + Active Directory/LDAP Integration

An SSL VPN server (ocserv) was deployed on Monitor-01, integrated with the IB.local Active Directory via PAM-LDAP. Domain users authenticate to the VPN using their standard AD credentials. This replicates the same principle used by enterprise products such as FortiClient with LDAP/AD integration.

Architecture

```
Internet/Client --> ocserv (Monitor-01 :443)
|
v LDAP authentication
DC01 :389 (IB.local)
|
v success
VPN tunnel active
IP range: 10.10.10.0/24
DNS: 192.168.1.10 (DC01)
Route: 192.168.1.0/24
```

ocserv Installation

```
sudo apt install -y ocserv ldap-utils
ocserv --version
# OpenConnect VPN Server 1.2.4
# Compiled with: seccomp, oath, radius, gssapi, PAM, PKCS#11, AnyConnect
```

LDAP Connectivity Test

Before configuring the VPN, LDAP connectivity to DC01 was verified using ldapsearch from Monitor-01.

```
ldapsearch -x -H ldap://192.168.1.10 \
-D "IB\Administrateur" -w "NewAdmin@2026" \
-b "DC=IB,DC=local" "(sAMAccountName=a.admin)" cn

# Result:
# dn: CN=Ayman Admin,OU=IT,OU=IB-Company,DC=IB,DC=local
# cn: Ayman Admin
# result: 0 Success
```

TLS Certificate Generation

A self-signed certificate was generated using GnuTLS certtool for TLS termination on the VPN endpoint.

```
sudo mkdir -p /etc/ocserv/ssl
cd /etc/ocserv/ssl

# Generate private key (3072-bit RSA)
sudo certtool --generate-privkey --outfile server-key.pem

# Create certificate template
cat > server.tmpl << EOF
cn = "IB VPN"
organization = "IB-Company"
expiration_days = 3650
signing_key
tls_www_server
EOF

# Generate self-signed certificate
sudo certtool --generate-self-signed \
```

```
--load-privkey server-key.pem \  
--template server.tmpl \  
--outfile server-cert.pem
```

ocserv Configuration

```
# /etc/ocserv/ocserv.conf  
auth = "pam" # PAM-LDAP authentication  
  
tcp-port = 443  
udp-port = 443  
run-as-user = ocserv  
run-as-group = ocserv  
  
server-cert = /etc/ocserv/ssl/server-cert.pem  
server-key = /etc/ocserv/ssl/server-key.pem  
  
ipv4-network = 10.10.10.0  
ipv4-netmask = 255.255.255.0  
dns = 192.168.1.10 # DC01 for domain name resolution  
route = 192.168.1.0/255.255.255.0  
  
max-clients = 10  
max-same-clients = 2  
device = vpns  
socket-file = /run/ocserv-socket
```

PAM-LDAP Configuration

```
# Install PAM-LDAP packages  
sudo apt install -y libpam-ldap nscd  
  
# Configure /etc/ldap.conf  
host 192.168.1.10  
base DC=IB,DC=local  
binddn CN=Administrateur,CN=Users,DC=IB,DC=local  
bindpw NewAdmin@2026  
pam_login_attribute SAMAccountName  
pam_filter objectclass=User  
nss_base_passwd DC=IB,DC=local?sub  
  
# Restart services  
sudo systemctl restart ocserv nscd
```

Service Status

```
sudo systemctl status ocserv  
  
# Output:  
# Active: active (running)  
# listening (TCP) on 0.0.0.0:443...  
# listening (UDP) on 0.0.0.0:443...  
# initialized OpenConnect VPN Server 1.2.4
```

```

PS C:\WINDOWS\system32> winget install --id=SimonTatham.Putty -e
The 'msstore' source requires that you view the following agreements before using.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
The source requires the current machine's 2-letter geographic region to be sent to the backend service to function properly (ex. "US").

Do you agree to all the source agreements terms?
[Y] Yes [N] No: yes
No package found matching input criteria.
PS C:\WINDOWS\system32> winget install --id=SimonTatham.Putty -e
No package found matching input criteria.
PS C:\WINDOWS\system32> curl.exe -k https://192.168.1.101/
<?xml version="1.0" encoding="UTF-8"?>
<config-auth client="vpn" type="auth-request">
<version who="sg">0.1(1)</version>
<auth id="main">
<message>Please enter your username.</message>
<form method="post" action="/auth">
<input type="text" name="username" label="Username:" />
</form></auth>
</config-auth>
PS C:\WINDOWS\system32> |

```

Figure 25 : ocserv responding to curl — AnyConnect auth-request page served on port 443

Client Connection Test

OpenConnect GUI was installed on Client-01 and configured to connect to the VPN server at 192.168.1.101. The self-signed certificate was accepted by the client, and the domain user a.admin authenticated successfully using Active Directory credentials via PAM-LDAP.

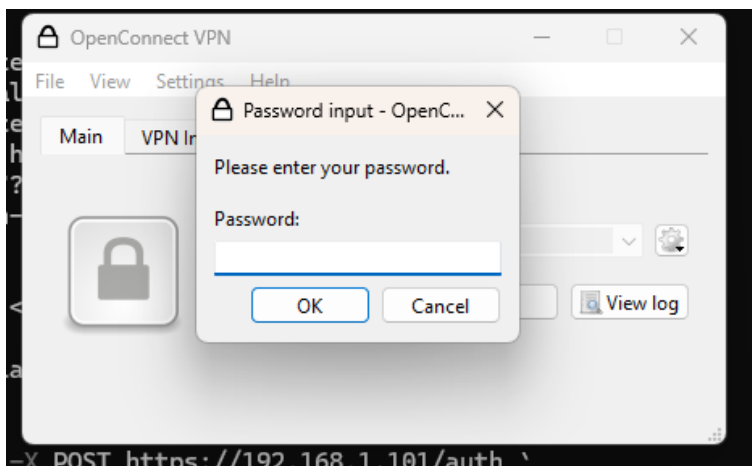


Figure 26 : OpenConnect VPN client showing green lock — connected to IB-VPN

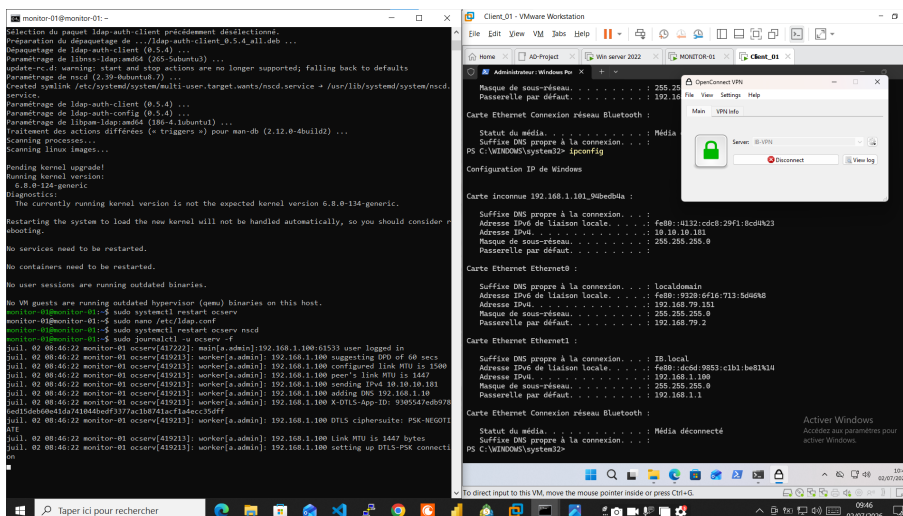


Figure 27 : Full connection proof — ocserv logs show a.admin logged in, Client-01 assigned IP 10.10.10.181, DNS 192.168.1.10

Connection Details

Parameter	Value
VPN Server	Monitor-01 at 192.168.1.101:443
Protocol	SSL/TLS — AnyConnect compatible
Authentication	PAM-LDAP against DC01 (IB.local)
Test account	a.admin (IB domain user)
IP assigned to client	10.10.10.181
DNS pushed to client	192.168.1.10 (DC01)
Route pushed	192.168.1.0/24 (full lab network accessible)
Certificate	Self-signed, GnuTLS 3072-bit RSA, 10-year validity

What's Next

The lab continues to grow. Each upcoming step adds a new enterprise layer on top of what is already running.

Graylog Dashboards

- : Widgets for top users with most failed logons
- : Login trend graphs per hour/day
- : Top event types by source machine
- : Alert history timeline

Windows Exporter in Grafana

- : Add DC01 metrics via Windows Exporter already installed on port 9182
- : CPU, RAM, disk, and network stats for the domain controller
- : Side-by-side comparison: Linux Monitor-01 vs Windows DC01

VPN Hardening

- : Replace self-signed certificate with a proper CA-signed certificate
- : Add two-factor authentication (TOTP) via ocserv oath module
- : Restrict VPN access to specific AD security groups (GRP-IT only)
- : Log VPN connections into Graylog via syslog

Conclusion

This lab demonstrates a complete, functional enterprise IT environment built from scratch on VMware Workstation. Every component was manually configured, scripted where possible, tested, and documented with evidence.

Section	Component	Status
1	Active Directory — DC01, IB.local, 6 OUs, 5 users, 6 GPOs, DHCP	Complete
2	Client-01 — Domain join, GPO enforcement verified	Complete
3	Monitoring — Prometheus + Node Exporter + Grafana live dashboard	Complete
4	Graylog — Docker Compose, NXLog, GPO software deploy, real-time logs	Complete
5	Graylog Alerts — Event ID 4625, aggregation, HTTP webhook	Complete
6	VPN SSL — ocserv + PAM-LDAP + AD authentication tested	Complete

Enterprise IT Lab — Ibnousoufyane Ayman

July 2026 | VMware Workstation | Windows Server 2022 · Windows 11 · Ubuntu 24.04

Stack: Active Directory · GPO · Prometheus · Grafana · Graylog · NXLog · ocserv VPN · PAM-LDAP

Repository: github.com/Ayman6594/AD-Project